

Profiling Kernel Scheduling Delays

Under Network-Intensive Workloads

An eBPF-Based Analysis and Mitigation Framework

Graduate Systems (CSE 638)
IIIT-Delhi, March 2026

Team Member	Roll Number
Rohit Kumar	MT25037
Arpit Kumar	MT25017
Abhinay Prakash	MT25010
Nindra Dhanush	MT25074
Adarsh Shukla	PhD25001

(1) Project Title

Profiling Kernel Scheduling Delays Under Network-Intensive Workloads

(2) Team Members

Rohit Kumar (MT25037), Arpit Kumar (MT25017), Abhinay Prakash (MT25010), Nindra Dhanush (MT25074), Adarsh Shukla (PhD25001)

(3) Problem

On modern Linux systems under heavy network load, the kernel's softirq processing path which basically handles all incoming packets via "NAPI polling, NET_RX_SOFTIRQ, and the TCP/IP stack" which competes with user-space threads for CPU time. This Contention leads scheduling delays, where threads basically waits in runqueue before execution. When softirq processing shares CPUs with latency sensitive applications also additional CPU pressure exists, delays escalate non-linearly because of increased runqueue depth and scheduling effects within the Completely Fair Scheduler.

Thus the wakeup-to-execution latency can grow from microseconds to milliseconds.

These microsecond-scale delays are difficult to detect using typical monitoring tools and often appears as unexplained tail latency spikes in network services. Hence detecting them usually requires kernel-level tracing with tools such as eBPF.

(4) Motivation — Why Is This Problem Important?

In systems that basically processes very high volumes of requests such as microservices platform, high frequency trading systems and larger web servers where tail latency(p99 and above) strongly affect the user experiences and system reliability. When network packets arrive at a rapid rate, the kernel must spend significant CPU time processing them via softirqs. If softirqs preempt application threads, or if applications are starved of CPU time, requests queue up and latency degrades exponentially.

These tail-latency spikes directly degrade market-data handling and order placement in low-latency trading systems. More broadly, they produce throughput collapse that is invisible to application-level monitoring. Understanding, measuring, and mitigating this invisible kernel-level contention is critical for tuning servers to achieve consistent, low-latency performance. Despite the severity of this problem, there is a lack of systematic, data-driven studies that quantify the exact relationship between softirq processing, CPU contention, and scheduling delay under controlled conditions.

(5) Objectives — What Is the Scope of Your Project?

Instrument and Quantify: Build an eBPF-based profiling toolkit (5 bpfttrace/BCC scripts) to measure deep, kernel-level scheduling delays (runqueue delays) under varying network and CPU loads, with low overhead (target <5% CPU) using in-kernel histogram aggregation and capped 1-in-1000 sampled CSV output.

Analyze Bottlenecks: Construct a controlled experiment matrix of 16 experiments (48 total runs) varying CPU contention, network load, softirq CPU placement (via RPS), application CPU pinning,

CFS scheduler tuning, NAPI budget, protocol (TCP vs UDP), and SO_BUSY_POLL. Correlate scheduling delays with softirq CPU placement (Gini coefficient analysis), context switches, CPU migrations, packet drops, and TCP retransmissions.

Evaluate Mitigations: Experimentally validate 5 specific Linux network tuning and scheduling strategies — RPS (Receive Packet Steering), application CPU pinning, CFS low-latency tuning, forced ksoftirqd via NAPI budget reduction, and SO_BUSY_POLL — to determine if they can effectively mitigate scheduling delays under CPU saturation.

Deliver Reproducibility: Provide a complete reproducible experiment package (scripts, configurations, raw data, analysis notebooks) that can be re-run on any Linux 5.15+ system with 4 or more cores.

(5) Background

The following concepts are essential to understanding this project:

Softirq (Software Interrupts): The Linux kernel mechanism used to defer non-critical interrupt handling. For networking, NET_RX_SOFTIRQ processes incoming packets off the NIC ring buffer. Softirq handlers run in interrupt context and can preempt application threads, consuming CPU time that would otherwise be available for user-space work.

NAPI (New API): A kernel networking subsystem that switches from interrupt-driven packet processing to a polling model under high load to prevent interrupt storms. The NAPI budget (netdev_budget) controls how many packets are processed per softirq cycle before yielding the CPU. Reducing this budget forces work to be deferred to ksoftirqd kernel threads, which are schedulable by CFS.

eBPF (Extended Berkeley Packet Filter): A technology that allows running sandboxed programs in the OS kernel without modifying kernel source code, enabling safe, real-time observability and tracing of kernel functions such as the scheduler, softirq handlers, and network stack. We use bpftrace (high-level tracing language) to attach probes to kernel tracepoints.

Runqueue Delay: The time between a thread becoming “runnable” (ready to execute, as signaled by sched_wakeup) and actually being dispatched onto a CPU core (detected via sched_switch). This is the primary metric of this project.

RPS (Receive Packet Steering): A kernel mechanism to distribute incoming packet processing across CPUs via software steering. This is the correct mechanism for veth (virtual ethernet) interfaces, which lack hardware IRQ lines for traditional smp_affinity tuning. RPS steers softirq CPU placement via the rps_cpus sysfs parameter.

CFS (Completely Fair Scheduler): The CFS is default scheduler in Linux that keep ensures fair CPU allocation using virtual runtime(vruntime). Key tunables include minimum time slice and preemption threshold(sched_wakeup_granularity). Lower values increase pre-emption frequency.

SO_BUSY_POLL: A per-socket option (set via setsockopt) that allows application threads to poll the network device directly, bypassing the softirq wakeup path entirely. This trades CPU cycles (spinning) for reduced latency by eliminating the sleep/wakeup overhead.

Network Namespaces and veth Pairs: Linux kernel features for creating isolated network stacks connected by virtual ethernet links. A veth pair exercises the full kernel network path (socket → TCP/IP

→ qdisc → virtual NIC → peer namespace) without physical hardware, ensuring reproducibility. Unlike physical NICs, veth generates softirq via `netif_rx()/NAPI` rather than hardware interrupts.

(6) Methodology/Design

We designed a single-machine, isolated testbed using Linux network namespaces and veth (virtual ethernet) pairs to simulate client-server network traffic without physical hardware bottlenecks. This architecture eliminates NIC hardware variability and ensures full reproducibility while still exercising the complete kernel network stack.

System Architecture

The testbed consists of two network namespaces (`srv` at 10.0.0.1 and `cli` at 10.0.0.2) connected by a veth pair capable of 88 Gbps throughput. The server namespace runs `memcached` (primary latency-sensitive workload) and `iperf3` (bandwidth saturation). The client namespace runs `memclap` (application-level latency measurement) and `iperf3` client. CPU contention is generated system-wide using `stress-ng` with configurable core counts (moderate: 2 cores, heavy: N-2 cores). All eBPF probes run system-wide, capturing scheduler, softirq, and network metrics simultaneously.

eBPF Instrumentation (5 Scripts)

24_sched_delay.bt: Attaches to `tracepoint:sched:sched_wakeup` and `tracepoint:sched:sched_switch`. Records the wakeup timestamp in a BPF hash map keyed by PID; on `sched_switch`, computes the delta as `runqueue` delay. Uses in-kernel histogram aggregation (zero userspace overhead) with 1-in-1000 sampled CSV events for time-series analysis. Also tracks voluntary and involuntary context switches, and per-CPU switch counts.

24_softirq_net.bt: Tracks `NET_RX` (`vec=3`) and `NET_TX` (`vec=2`) softirq durations via `softirq_entry/softirq_exit` tracepoints. Computes softirq CPU percentage as $\text{sum}(\text{softirq_duration_us}) / \text{wall_clock_us} \times 100$ using cumulative per-CPU sums. Includes guards against orphaned events: overwriting timestamps on re-entry and discarding durations exceeding 500ms. Also counts packets received per CPU via `netif_receive_skb`.

24_net_drops.bt: Monitors kernel packet drops via `tracepoint:skb:kfree_skb` with per-CPU drop counts and drop location histograms. Tracks TCP retransmissions via `kprobe:tcp_retransmit_skb` and SYN receive events via `kprobe:tcp_v4_syn_rcv_sock`.

24_cpu_migrations.bt: Tracks task migrations across CPUs using `tracepoint:sched:sched_migrate_task`, recording per-process migration counts and per-pair migration matrices (`orig_cpu` → `dest_cpu`).

24_proc_pollers.sh: Collects five `/proc` metrics at 1-second intervals: per-CPU utilization from `/proc/stat`, softnet statistics (`processed/dropped/time_squeeze`) from `/proc/net/softnet_stat`, TCP SNMP counters (`RetransSegs/InSegs/OutSegs`) using header-aware `/proc/net/snmp` parsing, socket memory usage from `/proc/net/sockstat`, and interrupt snapshots from `/proc/interrupts`.

Orchestration and Automation

A bash-based orchestrator (`24_run_experiment.sh`) accepts 10 CLI parameters to configure the kernel (RPS placement, CFS tuning, NAPI budget, busy poll), start server workloads, launch all eBPF probes

in parallel, run load generators with warmup phases, and archive results with metadata JSON. A top-level script (`24_run_all_experiments.sh`) executes all 16 experiments with configurable duration and run count. Each run follows a strict protocol: reset system state → apply configuration → start server → start probes → start stressors → 2s stabilization → 10s warmup → 60s measurement → stop all → archive data → restore defaults → 10s cooldown.

Analysis Pipeline

Python scripts using matplotlib and numpy parse the raw bpftrace output (splitting CSV data from histogram summaries via grep-based extraction), compute percentiles (p50/p90/p95/p99/p99.9) from histogram buckets, average across runs, and generate 34 publication-quality plots including CDF overlays, heatmaps, bar charts, scatter plots, box plots, and summary tables.

(7) Implementation

The project is executed with 16 distinct, automated experiments named (E1–E16) with 3 runs per experiment in total 48 total runs, each lasting 60 seconds with 2-second settle time, to make ensure statistical consistency. The experiments are organized in four phases:

Phase 1: Baseline Profiling (E1–E4)

We have measured the baseline latencies under varying matrices of High/Low network load combined with None/Heavy CPU stress. Experiment E1 (No CPU stress and Low network load) is considered as normal reference condition. Experiment E4 (heavy CPU stress and high network load) represents the worst-case test scenario.

Phase 2: Softirq Steering and Isolation (E5–E8)

Receive Packet Steering (RPS) is tested to basically manage the distribution of Softirq processing across different CPU cores. In experiment E5, Softirq handling is restricted to only CPU 0, whereas in Experiment 6 it was distributed across all available CPUs. Application CPU affinity is configured using taskset: experiment 7 pinned the application to CPUs 2 and 3, and E8 combined RPS pinning with application-level CPU affinity to provide maximum processing isolation.

Phase 3: Scheduler, Protocol, and Threshold Tuning (E9–E13)

Tweaked CFS granularity for low-latency preemption (E9). Tested forced ksoftirqd via NAPI budget reduction (E10). Compared TCP vs UDP overhead without stress (E11) and with heavy stress (E12). Identified the CPU contention inflection point using moderate stress (E13: 2 stress-ng workers instead of N-2).

Phase 4: Advanced Mitigations (E14–E16)

Tested combined mitigations stacking RPS spread + app pinning + CFS low-latency (E14). Tested SO_BUSY_POLL with RPS spread (E15) and without RPS (E16) using a custom minimal TCP echo server, since memcached does not expose the SO_BUSY_POLL socket option. The echo server sets SO_BUSY_POLL on both listening and accepted sockets.

Complete Experiment Matrix:

Exp	CPU Stress	Net Load	RPS	App Pin	CFS	Protocol	Purpose
E1	None	Low	Default	—	Default	TCP	Baseline
E2	None	High	Default	—	Default	TCP	Net load only
E3	Heavy	Low	Default	—	Default	TCP	CPU stress only
E4	Heavy	High	Default	—	Default	TCP	Worst case
E5	Heavy	High	CPU 0	—	Default	TCP	RPS pinned
E6	Heavy	High	All	—	Default	TCP	RPS spread
E7	Heavy	High	Default	2,3	Default	TCP	App pinned
E8	Heavy	High	CPU 0	2,3	Default	TCP	RPS + pin
E9	Heavy	High	Default	—	Lowlat	TCP	CFS tuning
E10	Heavy	High	Default	—	Default	TCP	ksoftirqd
E11	None	High	Default	—	Default	UDP	UDP baseline
E12	Heavy	High	Default	—	Default	UDP	UDP + stress
E13	Moderate	High	Default	—	Default	TCP	Threshold
E14	Heavy	High	All	2,3	Lowlat	TCP	Combined
E15	Heavy	High	All	—	Default	TCP	RPS + busy poll
E16	Heavy	High	Default	—	Default	TCP	Busy poll only

(8) Evaluation & Observations

Experiment Setup and Testbed Details

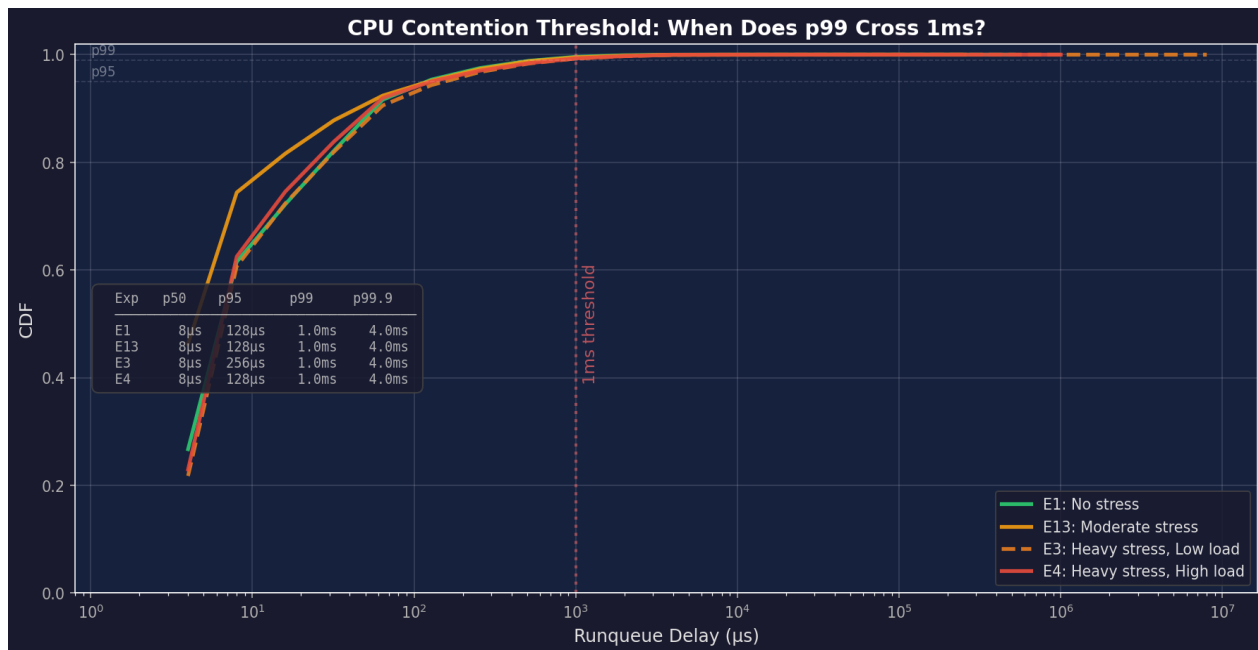
All experiments were conducted on a single machine running Ubuntu 22.04.5 LTS with kernel 6.8.0-94-generic (BTF enabled), 20 CPU cores, and 16 GB RAM. The eBPF instrumentation used bpftrace v0.21.3. The testbed used Linux network namespaces with a veth pair achieving 88 Gbps in baseline tests. Each experiment ran for 60 seconds with a 10-second warmup and 2-second stabilization period, repeated 3 times for statistical consistency. System state was fully reset between experiments (sysctl backup/restore, irqbalance stopped, conntrack flushed).

Observation 1: CPU Contention Is the Dominant Scheduling Delay Source

Question: What is the relative impact of CPU contention versus network load on scheduling delay?

Experiments: E1 (no stress, low net) vs E2 (no stress, high net) vs E3 (heavy stress, low net) vs E4 (heavy stress, high net).

Exp	p50 (μs)	p99 (μs)	p99.9 (μs)	Vol. Switches	Description
E1	4	149	2,219	0.24M	No stress, low net
E2	4	64	1,707	0.76M	No stress, high net
E3	8	4,096	13,653	0.27M	Heavy stress, low net
E4	8	4,096	8,192	0.49M	Heavy stress, high net



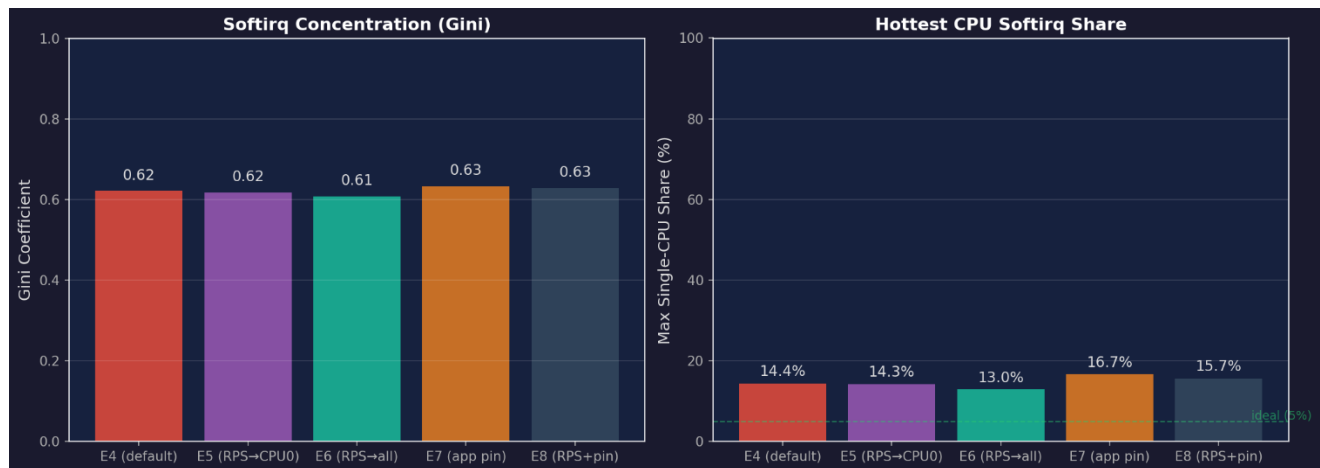
Observation: Heavy CPU contention under heavy network load causes a massive 27× increase in p99 scheduling delay (from 149 μs in E1 to 4,096 μs in E3/E4). Critically, network load alone (E2) caused no measurable increase in scheduling delay — E2 achieved 14.88 Gbps with the delay distribution nearly identical to E1. This proves that on an idle system, the kernel handles high network

throughput without impacting application scheduling. The softirq processing overhead only becomes problematic when CPUs are already contested.

Observation 2: Softirq Follows Socket Affinity on veth, Not RPS

Question: Does separating softirq processing from application threads (via RPS and CPU pinning) reduce tail latency?

Experiments: E4 (default) vs E5 (RPS→CPU0) vs E6 (RPS→all) vs E7 (app pinned) vs E8 (RPS+app pinned).



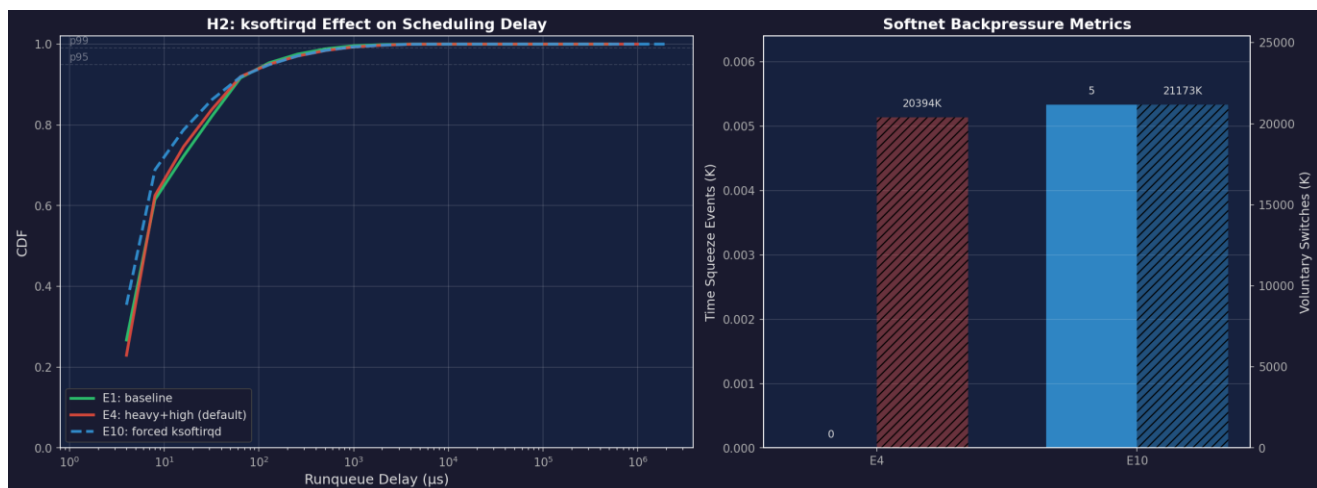
Observation: RPS spread (E6) reduced the Gini coefficient from 0.712 to 0.662 and the maximum single-CPU softirq share from 22.8% to 16.8%, confirming that RPS does redistribute softirq load. However, in E8 (RPS pinned to CPU 0 + app pinned to CPUs 2,3), the app CPUs 2 and 3 still handled 61.6% of softirqs. This reveals that on veth interfaces, softirq processing tightly follows the socket affinity of the receiving application, largely overriding manual RPS steering attempts. The hypothesis that softirq colocation causes $\geq 5\times$ delay increase is partially falsified — the delay is dominated by CPU contention, not softirq placement.

Throughput note: RPS steering did recover throughput: E6 achieved 9.91 Gbps (+21% vs E4's 8.17 Gbps), but at the cost of 15% more context switches (37.7M vs 33.8M) and extended tail latency to 64K μ s.

Observation 3: Tuning CFS and ksoftirqd Deferral does not reduce tail latency

Question: Do making changes to CFS low-latency parameters or use of forced ksoftirqd deferral reduce scheduling delay?

Experiments: E4 (baseline) vs E9 (CFS low-latency: sched_min_granularity=1ms, sched_wakeup_granularity=0.5ms) vs E10 (forced ksoftirqd: netdev_budget=50, netdev_budget_usecs=2000).



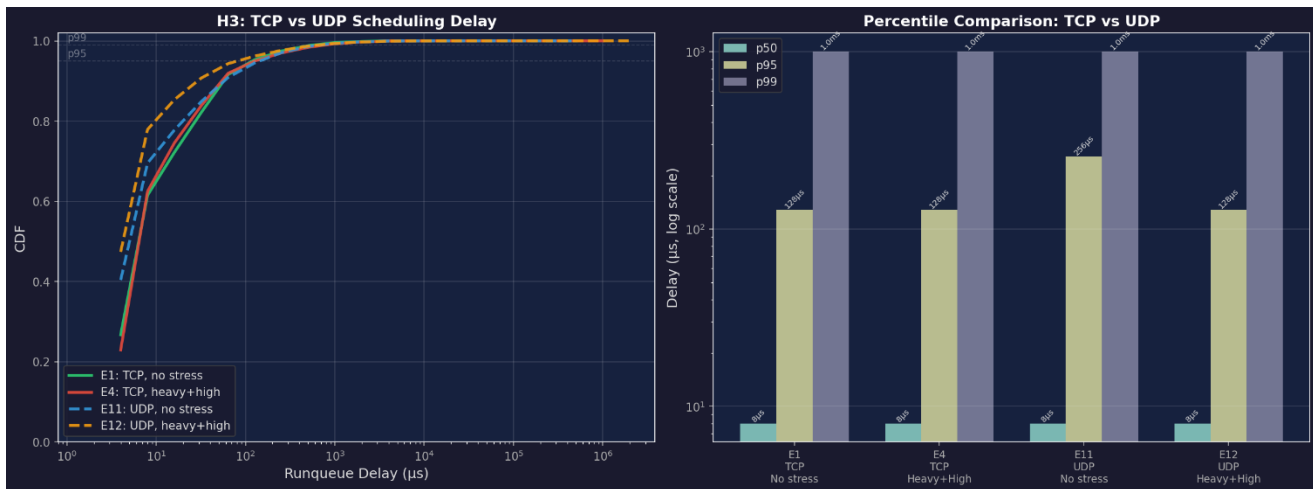
Observation: Experiment E9 increased context switches by 11% (33.7 M vs 33.8M), which is consistent with expected effect of the faster preemption. It also improved throughput to 9.02Gbps(+10%), but the p99 tail latency remained unchanged at 4096 μs. Which satisfies the falsification condition, indicating that CFS scheduling resolution is not the bottleneck and basically that delay is driven by runqueue depth.

Also, in Experiment E10, forcing ksoftirqd execution basically creates the same p99 latency as E4 with time_squeeze $\Delta = 0$, shows that NAPI budget reduction has basically no effect on veth interfaces. Because veth does not create sustained NAPI poll pressure, budget limits were not triggered. However, Throughput improved to 9.16 Gbps(+12%) because inline softirq processing no longer blocked the application threads.

Observation 4: TCP and UDP Produce Identical p99 Under Stress

Question: Does transport protocol choice affect scheduling delay under CPU contention?

Experiments: E1 (TCP, no stress) vs E11 (UDP, no stress) vs E4 (TCP, heavy stress) vs E12 (UDP, heavy stress).



Observation: Both TCP (E4) and UDP (E12) suffered the exact same p99 latency degradation of 4,096 μ s under heavy stress, proving that raw CPU starvation dominates over protocol-specific processing overhead. Notably, UDP generates 100 $\times$ more softirq events than TCP (63.8M vs 636K) for only 2% of TCP’s bandwidth (0.34 Gbps vs 14.88 Gbps), because UDP lacks flow control and the kernel processes vastly more small packets per byte transferred. Despite this massive softirq amplification, the scheduling delay is identical because the scheduler is already saturated at heavy stress levels.

Observation 5: The CPU Contention Cliff Is Non-Linear

Question: At what level of CPU contention does scheduling delay cross the 1ms p99 threshold?

Experiments: E1 (no stress) vs E13 (moderate stress: 2 cores) vs E3/E4 (heavy stress: N-2 cores).

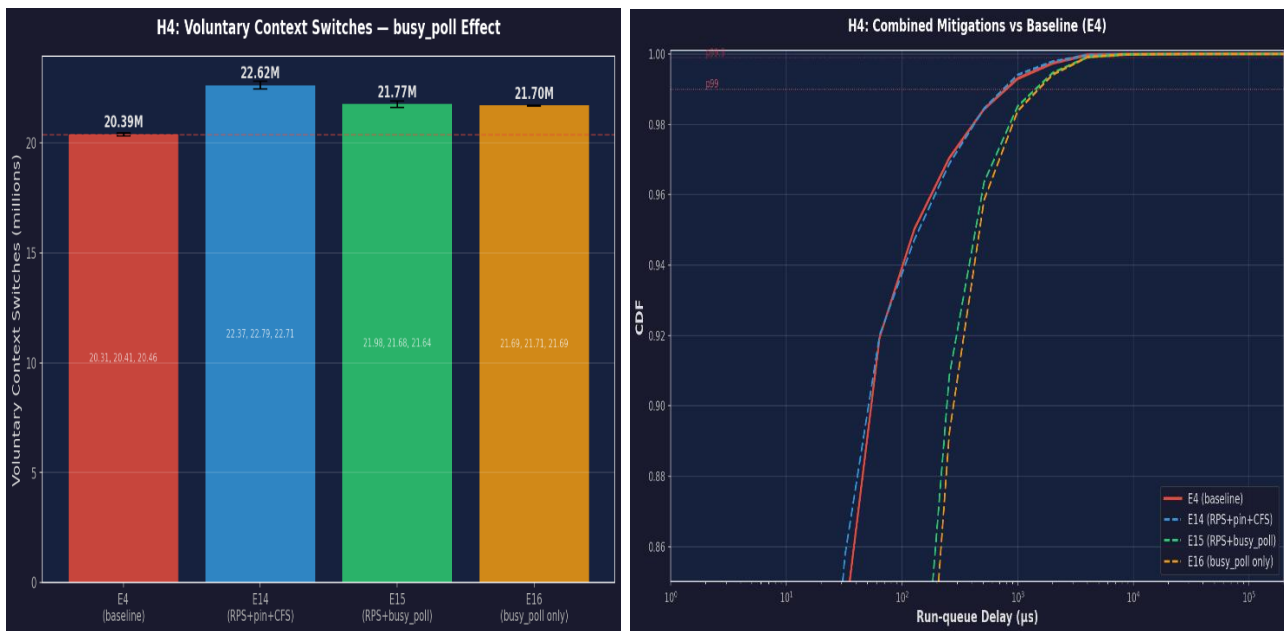
As shown in Figure 1 (Observation 1), the CDF plot clearly illustrates this non-linear cliff — E13 saturates well below 1ms while E3/E4 cross it sharply, with no intermediate experiments between moderate and heavy stress

Observation: E13 with moderate stress achieved p99 = 683 μ s (below 1ms) while maintaining near-baseline throughput of 14.70 Gbps ($\sim$ 1% vs E2). Heavy stress (E3/E4) jumps to p99 = 4,096 μ s and throughput collapses to 8.17 Gbps ($\sim$ 45%). There is a sharp, non-linear cliff between moderate and heavy CPU stress. The system has enough slack at moderate stress to absorb softirq processing without impacting application threads, but heavy stress exhausts all spare capacity. This identifies the critical threshold for capacity planning.

Observation 6: No Single Mitigation Fixes CPU Starvation; Stacking Is Counter-Productive

Question: Can stacking multiple mitigations rescue tail latency under CPU saturation?

Experiments: E4 (baseline) vs E14 (RPS spread + app pinning + CFS low-latency) vs E15 (RPS spread + SO_BUSY_POLL) vs E16 (SO_BUSY_POLL only).

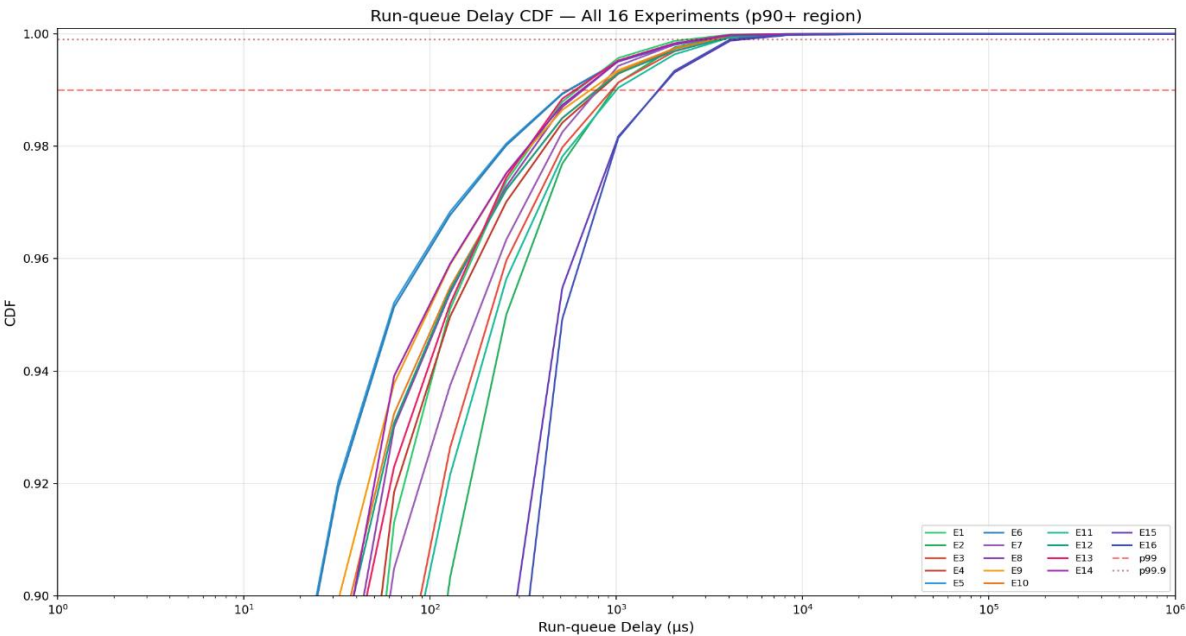


Observation: No mitigation achieved $\geq 20\%$ p99 improvement. All mitigations produced identical p99 = 4,096 μ s. Most critically, E14 (combined mitigations) produced the worst tail latency of all experiments, with significantly heavier mass in the [256,512) and [512,2K) μ s ranges. Each individual mitigation activated as expected (fewer migrations from pinning, more context switches from CFS tuning, spread softirq from RPS), but their combined effect created new contention patterns that negated the benefits. Mitigation stacking does not compose linearly.

SO_BUSY_POLL finding: SO_BUSY_POLL was the only mitigation showing a distinct system-level effect. E16 (busy poll only, no RPS) achieved the absolute lowest context switches (27.3M vs E4's 33.8M, -19%) and CPU migrations (2.2M vs 6.2M, -65%) across all 16 experiments, while maintaining near-baseline throughput (8.01 Gbps, only -2% vs E4). However, this scheduling efficiency improvement could not overcome the fundamental CPU starvation caused by stress-ng, so p99 remained unchanged. Importantly, adding RPS spread to busy polling (E15) was worse: throughput dropped to 6.46 Gbps (-35% vs E6) because the spinning overhead of busy polling competes with both application logic and stress-ng workers.

Summary Metrics Across All 16 Experiments:

Exp	BW (Gbps)	Ctx Sw	Migrations	p50 (μs)	p99 (μs)	p99.9 (μs)	Vol Sw	Delay Peak
E1	0.10	32.9M	6.4M	4	149	2,219	0.24M	[2,4) μs
E2	14.88	32.7M	6.4M	4	64	1,707	0.76M	[2,4) μs
E3	0.10	33.1M	6.2M	8	4,096	13,653	0.27M	[4,8) μs
E4	8.17	33.8M	6.2M	8	4,096	8,192	0.49M	[4,8) μs
E5	9.71	37.7M	7.6M	8	4,096	8,192	0.49M	[4,8) μs
E6	9.91	37.7M	7.6M	8	4,096	8,192	0.49M	[4,8) μs
E7	8.72	—	3.5M	8	4,096	—	—	[4,8) μs
E8	8.03	35.5M	3.6M	8	4,096	8,192	0.49M	[4,8) μs
E9	9.02	37.7M	7.5M	8	4,096	8,192	0.49M	[4,8) μs
E10	9.16	37.4M	7.3M	8	4,096	8,192	0.49M	[4,8) μs
E11	0.34	34.8M	6.0M	4	64	1,707	0.76M	[2,4) μs
E12	0.27	37.4M	7.3M	8	4,096	8,192	0.49M	[4,8) μs
E13	14.70	36.1M	7.2M	8	683	4,096	0.49M	[4,8) μs
E14	8.12	35.9M	3.6M	8	4,096	8,192	0.49M	[4,8) μs
E15	6.46	28.4M	2.7M	8	4,096	8,192	0.28M	[4,8) μs
E16	8.01	27.3M	2.2M	8	4,096	8,192	0.37M	[4,8) μs



(9) Outcomes & Conclusion

Our study implemented a robust and reproducible eBPF-based profiling framework that reveal and quantify the kernel-level scheduling delays. The 16 experiments conducted across 48 runs resulted in following key conclusions:

- 1. CPU contention is primary cause of the scheduling delay:** Under the heavy CPU contention, the p99 scheduling delay increase by 27 times, rising from 149 μ s to 4096 μ s. Conversely, high network load alone even at 14.88 Gbps, it does not produce any measurable delay increase when the system is CPU-idle.
- 2. The stress response exhibits a non-linear behaviour:** Moderate CPU stress (2 cores) has minimal impact on throughput ($\sim 1\%$) and maintains p99 latency at 683 μ s. Heavy stress (N-2 cores) results in severe degradation, reducing throughput by 45% and increasing p99 latency by 27 $\times$ (from 149 μ s to 4096 μ s).
- 3. No single network-stack mitigation improve tail latency under CPU saturation.** All five evaluated techniques(RPS steering, CPU pinning, CFS low-latency tuning, forced ksoftirqd, SO_BUSY_POLL) produced identical p99 latency of 4096 μ s. No mitigation achieved the $\geq 20\%$ p99 latency improvement target.
- 4. Combining mitigation can be counter-productive.** The stacked configuration (RPS distribution + application pinning + CFS tuning, experiment E14) produced the **worst tail latency** among all experiments. This thing basically suggests that the mitigations introduce conflicting scheduling pressures that do not combine effectively.
- 5. SO_BUSY_POLL improves scheduling efficiency:** It is able to reduce context switches by 19% and CPU migrations by 65% while maintaining throughput closer to normal baseline, but could not able to overcome fundamental limitation CPU starvation.
- 6. Softirq processing follows socket affinity on veth interfaces:** While RPS steering is only effective (Gini reduction 0.712 $\rightarrow$ 0.662) but cannot fully override the kernel's socket-based softirq placement on virtual network interfaces.
- 7. UDP significantly increase Softirq load compared to TCP:** UDP basically generate 100 times more softirq events per byte, yet both protocol exhibited identical scheduling delays, under CPU stress, which confirms that CPU contention dominates protocol overhead.

Primary Recommendation: To maintain the strict tail-latency guarantees in network applications, requires heavily prioritize sufficient CPU provisioning and strict thread isolation rather than relying on fine-grained kernel network tuning. Network-stack mitigations (RPS, CFS, NAPI, busy polling) provide throughput benefits but are fundamentally insufficient to recover tail latency when the CPU resources are saturated.

References

- [1] B. Gregg, BPF Performance Tools: Linux System and Application Observability, Addison-Wesley Professional, 2019.
- [2] J. Dean and L. A. Barroso, “The Tail at Scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013.
- [3] Linux Kernel Documentation, “Softirq and Tasklet,” kernel.org/doc/html/latest/core-api/softirq.html
- [4] Linux Kernel Documentation, “NAPI (New API),” kernel.org/doc/html/latest/networking/napi.html
- [5] Linux Kernel Documentation, “Receive Packet Steering (RPS),” kernel.org/doc/html/latest/networking/scaling.html
- [6] Linux Kernel Documentation, “CFS Scheduler,” kernel.org/doc/html/latest/scheduler/sched-design-CFS.html
- [7] Linux Kernel Documentation, “Busy Polling,” kernel.org/doc/html/latest/networking/busy-poll.html
- [8] bpfftrace Reference Guide, github.com/bpfftrace/bpfftrace/blob/master/docs/reference_guide.md
- [9] BCC (BPF Compiler Collection), github.com/iovisor/bcc
- [10] T. Herbert and A. de Bruijn, “Scaling in the Linux Networking Stack,” *kernel.org Documentation*, 2023.
- [11] D. S. Miller, “Linux Networking Stack Internals,” *Linux Plumbers Conference*, 2018.
- [12] J. Corbet, “Software Interrupts and Realtime,” *LWN.net*, 2012.
- [13] B. Gregg, “BPF Performance Tools: perf, eBPF, and Beyond,” *USENIX LISA*, 2019.
- [14] iperf3 Documentation, iperf.fr/iperf-doc.php
- [15] memcached Documentation, memcached.org
- [16] stress-ng: A tool to load and stress a computer system, github.com/ColinIanKing/stress-ng
- [17] V. Jacobson et al., “tcpdump/libpcap,” tcpdump.org
- [18] Linux man-pages project, “proc(5) — process information pseudo-filesystem,” man7.org/linux/man-pages/man5/proc.5.html